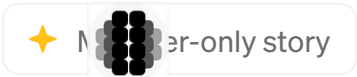


Generative AI



Stay updated with the latest news, research, and developments in the world of generative AI. We cover everything from AI model updates, APIs, WebSockets, and plain HTML/JS for a local Ollama agent — tutorials, and real-world applications to the broader impact of AI on society. Work with us: jimclydegm@gmail.com

Build Your Own Claude Code Web UI in 280 Lines of Python

FastAPI, WebSockets, and plain HTML/JS for a local Ollama agent — with sessions, skills, and context compaction.



Jes Fink-Jensen

jimclydegm@gmail.com

Follow

20 min read · 5 days ago

Following

 70















In Part 1 of this series, I showed you how to build a local LLM agent in Python with Ollama. We went from a basic chat loop to a full agent with tools, skills, slash commands, history persistence, context compaction, and background loops, all in a single Python file.

So far, the agent runs in the terminal. This is fine for development, but it is also a bit awkward to demo, and the thinking-versus-answer distinction gets lost in a wall of grey text. Therefore, in this article, I will show you how to wrap the agent in a small FastAPI server, expose it over a WebSocket, and build a plain HTML+JS frontend that streams the response token-by-token to the browser.

The result is a two-pane chat UI with the conversation on the right and the controls (sessions, skills, slash commands) on the left. No frontend framework, no build step, no npm. Just one Python file and one HTML file.

Here are the seven stages we will go through:

- Stage 1: Refactoring the agent into a reusable class
- Stage 2: A minimal FastAPI server with a WebSocket endpoint
- Stage 3: The HTML+JS frontend
- Stage 4: Streaming thinking and content to the browser
- Stage 5: Session management in the UI
- Stage 6: A skills picker in the sidebar
- Stage 7: Context warnings, compaction, and slash commands as buttons

So, let us get started.

Installation

The full code is on GitHub at [jfjensen/building-a-local-LLM-agent-with-WebUI](https://github.com/jfjensen/building-a-local-LLM-agent-with-WebUI). Each stage lives in its own subdirectory and is exposed as a console script via `pyproject.toml`, so you can install once and then run any stage by name.

First, clone the repo and install it in editable mode:

```
git clone https://github.com/jjfjensen/building-a-local-LLM-agent-with-WebUI.git
cd building-a-local-LLM-agent-with-WebUI
pip install -e .
```

This pulls in FastAPI, Uvicorn, the `websockets` library, and the Ollama Python client, and registers seven console scripts: `chat-ui-stage1` through `chat-ui-stage7`. Each one starts the corresponding stage from the current working directory.

Then, make sure you have an Ollama model pulled. The default is `qwen3.5:9b`. If you want a different one, change `MODEL_NAME` at the top of the `main.py` for the relevant stage:

```
ollama pull qwen3.5:9b
```

Each stage creates `history/` and `skills/` folders in the current working directory on first run. So, if you want to keep things separate, it is best to run each stage from its own folder. For example:

```
mkdir my-session && cd my-session
chat-ui-stage7
```

So, with the install out of the way, let us go through the stages.

Stage 1: Refactoring the agent into a reusable class

First, we need to deal with a small problem from Part 1: the agent state lives inside the `chat()` function and a handful of module-level globals. This is fine for a single CLI session, but it does not work when we want to serve multiple browser tabs at the same time, each with its own session.

So, the first thing to do is to pull the state and behavior into a class. Let's call it `Agent`.

```
import ollama
import os
import json
from datetime import datetime

MODEL_NAME = 'qwen3.5:9b'
HISTORY_DIR = "history"
SKILLS_DIR = "skills"
CONTEXT_THRESHOLD = 4000
class Agent:
    def __init__(self, session_id=None):
        self.session_id = session_id or datetime.now().strftime("%Y-%m-%d_%H-%M")
        self.history_file = os.path.join(HISTORY_DIR, f"{self.session_id}.json")
        self.messages = []
        self.active_skill_content = ""
        if os.path.exists(self.history_file):
            with open(self.history_file, 'r', encoding='utf-8') as f:
                self.messages = json.load(f)
```

Here is a step-by-step description of the above code:

The `__init__` method:

- Each `Agent` instance has its own `session_id`. If one is passed in, we use it. Otherwise, we generate one from the current datetime, exactly like in Part 1.
- The `messages` list and `active_skill_content` variable, which used to be globals, are now instance attributes. This means each agent has its own conversation history and its own loaded skill.
- If a history file already exists for this session ID, we load it. This is what lets the browser reconnect to an old session and pick up where it left off.

Next, we move the helper functions from Part 1 onto the class. I will not repeat the bodies here, since they are the same as before. The signatures become:

```
class Agent:
    # ... __init__ as above ...

    def estimate_tokens(self):
        text = "".join([str(m.get('content', '')) for m in self.messages])
        return len(text) // 4

    def save_history(self):
        # same body as save_history() in Part 1, but using self.messages
        # and self.history_file instead of globals
        ...

    def compact_history(self):
        # same body as compact_history() in Part 1, but using self.messages
```

```
        # and self.active_skill_content
        ...
    def handle_tools(self, tool_calls):
        # same body as handle_tools() in Part 1, but as a method
        ...
```

The only real change is that anything that used to read or write a global now reads or writes `self.something` instead. This is a mechanical translation, and the logic stays the same.

So, with this in place, we can create multiple agents in the same process:

```
a1 = Agent()
a2 = Agent()
a1.messages.append({'role': 'user', 'content': 'hello'})
print(a2.messages) # still empty
```

This is exactly what we need for the WebSocket server, where each connection gets its own agent.

To prove the class actually works before we get into the WebSocket stuff, I wrap a small CLI loop around it inside a function called `chat()`. This is also the function that the `chat-ui-stage1` console script points to:

```
def chat():
```

```
agent = Agent()
print(f"--- Agent session: {agent.session_id} ---")
print("Type 'quit' to exit.\n")

while True:
    user_input = input("You: ").strip()
    if not user_input or user_input.lower() in ('quit', 'exit'):
        break

    agent.messages.append({'role': 'user', 'content': user_input})

    resp = ollama.chat(model=MODEL_NAME, messages=agent.messages, tools=tools)
    msg = resp['message']

    if hasattr(msg, 'tool_calls') and msg.tool_calls:
        agent.messages.append({'role': 'assistant', 'tool_calls': msg.tool_calls})
        final = agent.handle_tools(msg.tool_calls)
        print(f"Assistant: {final['content']}\n")
        agent.messages.append(final)
    else:
        print(f"Assistant: {msg.content}\n")
        agent.messages.append({'role': 'assistant', 'content': msg.content})

    agent.save_history()

if __name__ == "__main__":
    chat()
```

So, to run this stage:

```
chat-ui-stage1
```

Below you can see an example of such a run:

Screenshot of stage 1 running in a terminal

From stage 2 onwards, the same `chat()` function is what each console script calls, but instead of running a CLI loop it starts the FastAPI server. So, the naming is consistent across all seven stages.

Stage 2: A minimal FastAPI server with a WebSocket endpoint

Now we can build the server. The most straightforward way to stream tokens from Ollama to the browser is to use a WebSocket. The browser opens a single long-lived connection, and the server pushes messages over it as soon as they are ready. So, let us do that.

The FastAPI and Uvicorn dependencies are already installed from the `pip install -e .` step. So, we can go straight to writing a minimal server with two endpoints: one that serves the HTML page, and one that handles the WebSocket.

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.responses import FileResponse
import json
```

```

app = FastAPI()

@app.get("/")
async def root():
    return FileResponse("index.html")

@app.websocket("/ws/{session_id}")
async def websocket_endpoint(websocket: WebSocket, session_id: str):
    await websocket.accept()
    agent = Agent(session_id=session_id)
    try:
        while True:
            data = await websocket.receive_text()
            msg = json.loads(data)
            user_input = msg.get('content', '')
            agent.messages.append({'role': 'user', 'content': user_input})
            # We will fill this in during Stage 4
            await stream_to_websocket(agent, websocket)
            agent.save_history()
    except WebSocketDisconnect:
        print(f"[SYSTEM] Client disconnected from session {session_id}")

```

Three things to note:

- The session ID is part of the URL path. So, the browser decides which session it wants to connect to, and the server creates an `Agent` for that session. If a history file exists for that ID, the agent loads it; otherwise it starts fresh.
- The WebSocket protocol uses JSON for everything. The browser sends `{"content": "..."}` for a user message, and the server sends back messages with a `type` field (we will define the types in Stage 4).
- The `WebSocketDisconnect` exception is what FastAPI raises when the browser closes the tab or refreshes. We catch it so the server keeps

running.

Finally, we need a way to actually start the server. Instead of relying on the `uvicorn main:app --reload` command, we put the startup logic inside a `chat()` function, just like in Stage 1. This is what the `chat-ui-stage2` console script calls:

```
def chat():
    import asyncio
    import sys
    import uvicorn
    if sys.platform == "win32":
        asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())

    async def _serve():
        config = uvicorn.Config("chat_ui_02.main:app", host="127.0.0.1", port=8000)
        server = uvicorn.Server(config)
        await server.serve()

    try:
        asyncio.run(_serve())
    except KeyboardInterrupt:
        pass
```

A few things to note:

- Imports inside a function are lazily loaded — `uvicorn` only gets imported when `chat()` is actually called, not when the module is first imported by something else.
- We construct a `uvicorn.Config` and start the `Server` programmatically.

So, no shell command is needed; the console script just runs `chat()` .

- On Windows, we set the `WindowsSelectorEventLoopPolicy` . Otherwise, the default event loop policy on Windows does not play well with the way `websockets` does I/O, and you get some odd errors at shutdown.
- The module string `"chat_ui_02.main:app"` is what Uvicorn uses to find the FastAPI app instance. Each stage uses its own module name, e.g. `chat_ui_03.main:app` for stage 3, and so on.

So, to run this stage:

```
chat-ui-stage2
```

This starts a server on `http://127.0.0.1:8000/` . There is no UI yet, but you can test the WebSocket using `wscat` or a small Python one-liner:

```
python -c "import asyncio,websockets,json;exec('''async def m():\n ws=await web
```

You should get a result like this:

```
{"type":"echo","received":"hello","session_id":"test"}
```

Stage 3: The HTML+JS frontend

Next, we write the frontend. I want to keep this in a single file with no build step, so it is just plain HTML, CSS, and JavaScript. The page has two panes: a sidebar on the left for controls, and a chat area on the right.

Here is the skeleton:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Qwen Agent</title>
  <style>
    body { font-family: -apple-system, sans-serif; margin: 0; display: flex
    #sidebar { width: 240px; background: #f5f5f5; padding: 16px; overflow-y
    #main { flex: 1; display: flex; flex-direction: column; }
    #messages { flex: 1; overflow-y: auto; padding: 16px; }
    #input-row { display: flex; padding: 12px; border-top: 1px solid #ddd;
    #input { flex: 1; padding: 8px; font-size: 14px; }
    .msg { margin-bottom: 12px; }
    .msg.user { color: #1a73e8; }
    .msg.assistant { color: #222; }
    .msg.thinking { color: #888; font-style: italic; font-size: 13px;
      border-left: 2px solid #ccc; padding-left: 8px; }
    .msg.tool { color: #b8860b; font-family: monospace; font-size: 13px; }
    button { display: block; width: 100%; margin: 4px 0; padding: 6px;
      text-align: left; background: white; border: 1px solid #ddd;
    button:hover { background: #eee; }
    h3 { font-size: 12px; text-transform: uppercase; color: #666; margin-to
  </style>
</head>
<body>
  <div id="sidebar">
```

```
<h3>Sessions</h3>
<div id="sessions">(stage 5)</div>
<h3>Skills</h3>
<div id="skills">(stage 6)</div>
<h3>Commands</h3>
<div id="commands">(stage 7)</div>
</div>
<div id="main">
  <div id="messages"></div>
  <div id="input-row">
    <input id="input" placeholder="Type a message..." onkeydown="if(eve
    <button onclick="send()" style="width: auto; margin-left: 8px;">Sen
  </div>
</div>
<script>
  // We will fill this in during Stage 4
</script>
</body>
</html>
```

A few things to note:

- The whole thing fits on one screen. No build step, no bundler, no framework.
- The CSS is minimal on purpose. I want the chat content to be the focus, not the chrome around it.
- Each message type (user, assistant, thinking, tool) has its own CSS class with a different color. Thinking is grey and italic, tool calls are amber and monospaced. This is what makes the streaming visible at a glance.
- The sidebar has three sections: sessions, skills, and commands. We will fill these in during stages 5, 6, and 7.

If you run `chat-ui-stage3` from its subdir `\stage3` now and open `http://127.0.0.1:8000/` in your browser, you will see the layout, but nothing happens when you type.

Screenshot of the browser window in VSCode when running stage 3

So, the next thing is to wire up the WebSocket.

Stage 4: Streaming thinking and content to the browser

Now for the most interesting part: streaming the model output to the UI in

real time, with the thinking channel and the final answer rendered separately.

On the server side, we need a function that calls `ollama.chat` with `stream=True` and pushes each chunk over the WebSocket. This is essentially a rewrite of `stream_with_thinking` from Part 1, but instead of printing to `stdout`, it sends JSON messages.

```
async def stream_to_websocket(agent, websocket):
    response_stream = ollama.chat(
        model=MODEL_NAME, messages=agent.messages, stream=True, tools=tools,
    )

    full_content = ""
    collected_tool_calls = []

    await websocket.send_json({'type': 'start'})

    for chunk in response_stream:
        msg = chunk.message

        if hasattr(msg, 'tool_calls') and msg.tool_calls:
            collected_tool_calls = msg.tool_calls

        if hasattr(msg, 'thinking') and msg.thinking:
            await websocket.send_json({'type': 'thinking', 'content': msg.thinking})
        elif msg.content:
            await websocket.send_json({'type': 'content', 'content': msg.content})
            full_content += msg.content

    if collected_tool_calls:
        agent.messages.append({'role': 'assistant', 'tool_calls': collected_tool_calls})
        for tc in collected_tool_calls:
            await websocket.send_json({
                'type': 'tool_call',
                'name': tc.function.name,
                'args': tc.function.arguments,
```



```
    })
    final_msg = agent.handle_tools(collected_tool_calls)
    await websocket.send_json({'type': 'tool_result'})
    await websocket.send_json({'type': 'content', 'content': final_msg['con
    agent.messages.append(final_msg)
else:
    agent.messages.append({'role': 'assistant', 'content': full_content})

    await websocket.send_json({'type': 'end'})
```

Here is a step-by-step description:

The message types:

- `start`: tells the browser that a new assistant turn is beginning. The browser uses this to create a new message bubble.
- `thinking`: a chunk of the model's reasoning. The browser appends it to the current bubble, with the "thinking" CSS class.
- `content`: a chunk of the final answer. The browser appends it to the current bubble, with the "assistant" CSS class.
- `tool_call`: announces that the model is calling a tool. The browser renders this as a small monospaced line.
- `tool_result`: announces that the tool finished. We do not send the full result, since it could be huge; the browser just sees a placeholder.
- `end`: tells the browser the turn is over.

Handling tools:

- When the model decides to call a tool, we get a non-empty `collected_tool_calls`. We send a `tool_call` event for each one, then run `agent.handle_tools` to actually execute them, then send the final answer as a `content` event.
- This way, the user sees the agent “think”, then “use a tool”, then “answer”, all in sequence.

On the frontend, we open a WebSocket and dispatch incoming messages by type:

```
<script>
  let ws = null;
  let currentSessionId = null;
  let currentBubble = null;

  function connectWebSocket(sessionId) {
    if (ws) ws.close();
    currentSessionId = sessionId;
    ws = new WebSocket(`ws://${location.host}/ws/${sessionId}`);
    ws.onmessage = (event) => handleMessage(JSON.parse(event.data));
  }

  function handleMessage(data) {
    const messages = document.getElementById('messages');
    if (data.type === 'start' || data.type === 'end') {
      currentBubble = null;
    } else if (data.type === 'thinking') {
      appendToBubble('thinking', data.content);
    } else if (data.type === 'content') {
      appendToBubble('assistant', data.content);
    } else if (data.type === 'tool_call') {

```

```

        const line = document.createElement('div');
        line.className = 'msg tool';
        line.textContent = `→ ${data.name} (${JSON.stringify(data.args)})`;
        messages.appendChild(line);
        currentBubble = null;
    } else if (data.type === 'tool_result') {
        const line = document.createElement('div');
        line.className = 'msg tool';
        line.textContent = `← (tool executed)`;
        messages.appendChild(line);
        currentBubble = null;
    }
    messages.scrollTop = messages.scrollHeight;
}

function appendToBubble(cls, text) {
    const messages = document.getElementById('messages');
    if (!currentBubble || !currentBubble.classList.contains(cls)) {
        currentBubble = document.createElement('div');
        currentBubble.className = `msg ${cls}`;
        messages.appendChild(currentBubble);
    }
    currentBubble.textContent += text;
}

function send() {
    const input = document.getElementById('input');
    const text = input.value.trim();
    if (!text || !ws) return;

    const userDiv = document.createElement('div');
    userDiv.className = 'msg user';
    userDiv.textContent = text;
    document.getElementById('messages').appendChild(userDiv);

    ws.send(JSON.stringify({content: text}));
    input.value = '';
    currentBubble = null;
}

window.onload = () => connectWebSocket('default');
</script>

```

A few things to note:

- The `currentBubble` variable tracks the active message element. When a `thinking` chunk arrives, we either reuse the current bubble (if it is already a thinking bubble) or create a new one. Same for `content`. This is what gives the streaming effect: the bubble fills up as the chunks arrive, without flickering.
- When a tool call or tool result comes in, we reset `currentBubble` to `null`. This way, the next content chunk will start a fresh bubble below the tool line.
- The user message is added to the DOM immediately, before the WebSocket round-trip. So, the UI feels fast even on a slow model.

If you run `chat-ui-stage4` now and open the page, you should see something close to the screenshot below: a chat with the thinking in grey above the final answer, and tool calls shown inline in yellow.

Screenshot of the browser window running stage 4, showing request, thinking, tool use and final response

So, at this point we have the core feature working. The remaining three stages are about making the UI actually pleasant to use.

Stage 5: Session management in the UI

Currently the browser always connects to a session called `default`. So, every page load picks up the same conversation. This is fine for testing, but in practice we want to start new conversations and switch between old ones, the same way we did with `/history-list` and `/history-load` in Part 1.

First, we add an HTTP endpoint on the server that lists all saved sessions:

```
@app.get("/sessions")
async def list_sessions():
    if not os.path.exists(HISTORY_DIR):
        return {'sessions': []}
    files = sorted(os.listdir(HISTORY_DIR), reverse=True)
    sessions = [f.replace('.json', '') for f in files if f.endswith('.json')]
    return {'sessions': sessions}
```

This returns a JSON list of session IDs, newest first. Then we add a frontend function that fetches this list and renders it in the sidebar:

```
async function loadSessions() {
    const res = await fetch('/sessions');
    const data = await res.json();
    const container = document.getElementById('sessions');
    container.innerHTML = '';
    for (const sid of data.sessions) {
        const btn = document.createElement('button');
        btn.textContent = sid;
        btn.onclick = () => switchSession(sid);
        if (sid === currentSessionId) btn.style.background = '#dbe9ff';
        container.appendChild(btn);
    }
}

function switchSession(sid) {
    document.getElementById('messages').innerHTML = '';
    connectWebSocket(sid);
    loadSessions();
}

function newSession() {
    const sid = new Date().toISOString().replace(/[:.]/g, '-').slice(0, 19);
```

```
        switchSession(sid);  
    }
```

Here is a step-by-step description:

loadSessions :

- Fetches the list from `/sessions` .
- Clears the sidebar.
- For each session, creates a button. The currently active session gets a highlighted background.

switchSession :

- Clears the chat area.
- Opens a new WebSocket to the chosen session.
- Re-renders the sidebar so the highlight moves.

newSession :

- Generates a fresh session ID from the current datetime.
- Calls `switchSession` with the new ID.

We also need to load the past messages when reconnecting to a session, so the chat does not start empty. On the server side, we send the existing messages right after the WebSocket connects:

```
@app.websocket("/ws/{session_id}")
async def websocket_endpoint(websocket: WebSocket, session_id: str):
    await websocket.accept()
    agent = Agent(session_id=session_id)

    # Replay history to the browser
    for m in agent.messages:
        if m.get('role') == 'user':
            await websocket.send_json({'type': 'replay_user', 'content': m['con
        elif m.get('role') == 'assistant' and m.get('content'):
            await websocket.send_json({'type': 'replay_assistant', 'content': m
    try:
        while True:
            # ... same as before
            ...
```

On the frontend, we handle the two new message types by simply appending bubbles, without the streaming animation.

Finally, we call `loadSessions()` on page load, so the sidebar populates from the start.

Screenshot of the browser window running stage 5, showing a previous session loaded into the chat

Stage 6: A skills picker in the sidebar

In Part 1, the agent loaded skills by calling the `manage_skills` tool. So, the user had to say "what skills do you have?" and then "load the security auditor", and the model would do the rest. This works, but it wastes tokens, and it is also slow.

Therefore, in the UI, it makes more sense to expose skills directly. The user clicks a skill name in the sidebar, and the agent loads it instantly, without going through the model.

First, we add an HTTP endpoint to list skills:

```
@app.get("/skills")
async def list_skills():
    if not os.path.exists(SKILLS_DIR):
        return {'skills': []}
    skills = [f.replace('.md', '') for f in os.listdir(SKILLS_DIR) if f.endswith('.md')]
    return {'skills': skills}
```

Then we add a control message type to the WebSocket. The browser sends `{"control": "load_skill", "name": "humanizer"}`, and the server loads the skill into the agent.

```
# in the WebSocket loop:
while True:
    data = await websocket.receive_text()
    msg = json.loads(data)

    if 'control' in msg:
        if msg['control'] == 'load_skill':
            path = os.path.join(SKILLS_DIR, f"{msg['name']}.md")
            if os.path.exists(path):
                with open(path, 'r', encoding='utf-8') as f:
                    agent.active_skill_content = f.read()
                await websocket.send_json({
                    'type': 'system',
                    'content': f"Skill loaded: {msg['name']}",
                })
            else:
                await websocket.send_json({
                    'type': 'system',
                    'content': f"Skill not found: {msg['name']}",
                })
```

```
        continue

    user_input = msg.get('content', '')
    agent.messages.append({'role': 'user', 'content': user_input})
    ...
```

Three things to note:

- The control message is handled before the user input branch. So, control messages do not show up in the conversation history.
- After loading the skill, we send a `system` message back so the UI can show a small notice. I render this as a centered grey line in the chat area.
- We need to make sure the active skill is actually used when the model is called next. The simplest way is to inject it as a system message at the top of `agent.messages` whenever it is non-empty. This is a one-line change in `stream_to_websocket`, prepending `{'role': 'system', 'content': agent.active_skill_content}` to the message list passed to Ollama.

On the frontend, we fetch the skills and render buttons:

```
async function loadSkills() {
    const res = await fetch('/skills');
    const data = await res.json();
    const container = document.getElementById('skills');
    container.innerHTML = '';
```

```
    for (const skill of data.skills) {  
      const btn = document.createElement('button');  
      btn.textContent = skill;  
      btn.onclick = () => {  
        ws.send(JSON.stringify({control: 'load_skill', name: skill}));  
      };  
      container.appendChild(btn);  
    }  
  }  
}
```

And we call `loadSkills()` on page load, alongside `loadSessions()`.

So, now I can drop in a new `.md` file in the `skills/` folder, refresh the page, and the new skill shows up as a clickable button.

Screenshot of the browser window running stage 6, showing available skills and with the humanizer loaded

Stage 7: Context warnings, compaction, and slash commands as buttons

Finally, we wire up the remaining pieces from Part 1: the slash commands (`/context`, `/compact`, `/tools`), the auto-compaction logic, and a small new feature on top, which is context warnings.

The slash commands themselves are easy. In the CLI, they were shortcuts that ran some Python and printed the result. In the UI, the same thing, but as clickable buttons. The pattern is identical to the skills picker: a control

message over the WebSocket, handled before the user input branch.

So, let us deal with the easy ones first:

```
if 'control' in msg:
    ctrl = msg['control']
    if ctrl == 'load_skill':
        # ... as in Stage 6
        ...
    elif ctrl == 'tools':
        names = [t['function']['name'] for t in tools]
        await websocket.send_json({
            'type': 'system', 'content': f"Tools: {' ', ' '.join(names)}"
        })
    elif ctrl == 'context':
        tokens = agent.estimate_tokens()
        await websocket.send_json({
            'type': 'system', 'content': f"Context: {tokens}/{CONTEXT_THRESHOLD}"
        })
    elif ctrl == 'compact':
        # ... covered below
        ...
    continue
```

And on the frontend, the buttons in the sidebar just need a `sendControl` function:

```
function sendControl(name) {
    if (ws) ws.send(JSON.stringify({control: name}));
}
```

That is the simple part.

Async compaction

The more interesting part is `/compact`. In Part 1 and in Stage 1 of this article, `compact_history` was a regular synchronous method that called `ollama.chat` and waited for the result. This was fine in the CLI, but in a FastAPI WebSocket handler it is a problem: the call to `ollama.chat` is blocking, which means it blocks the entire asyncio event loop while the model is summarizing. So, the WebSocket cannot send or receive anything during that time, and the browser appears frozen.

The fix is to run the blocking call in a thread, using `asyncio.to_thread`. Here is how the agent looks now:

```
class Agent:
    def __init__(self, session_id=None):
        # ... same as before, plus:
        self._warned_thresholds = set()

    def _do_compact(self):
        """Blocking Ollama call - run via asyncio.to_thread."""
        split_idx = int(len(self.messages) * 0.7)
        to_summarize = self.messages[:split_idx]
        keep_fresh = self.messages[split_idx:]
        resp = ollama.chat(
            model=MODEL_NAME,
            messages=to_summarize + [{'role': 'user', 'content':
                "Summarize this conversation briefly, preserving key facts and
            }
        )
        summary = resp['message']['content']
        self.messages = [{'role': 'system', 'content': f"PREVIOUS SUMMARY: {sum
        self._warned_thresholds = set()
```

```
async def compact_history(self):
    if len(self.messages) < 4:
        return
    import asyncio
    try:
        await asyncio.to_thread(self._do_compact)
    except Exception as e:
        print(f"[ERROR] Compaction failed: {e}")
```

A few things to note:

- I split the method in two: `_do_compact` does the blocking work, and `compact_history` wraps it in `asyncio.to_thread` so the event loop stays responsive.
- The `compact_history` method is now `async`. So, callers need to `await` it.
- The `_warned_thresholds` set will become useful in a moment. I clear it inside `_do_compact` because once the context has been compacted, the warnings should start fresh.

The `/compact` button now looks like this on the server:

```
elif ctrl == 'compact':
    tokens = agent.estimate_tokens()
    await websocket.send_json({
        'type': 'system',
        'content': f"Compacting context... ({tokens}/{CONTEXT_THRESHOLD} tokens"
    })
    await agent.compact_history()
    await websocket.send_json({'type': 'system', 'content': 'Context compacted.'})
```


So, the user clicks the button, sees a “Compacting context...” message immediately, the server runs the summary in a thread, and then a “Context compacted.” message appears. The conversation can continue normally during this, and other concurrent sessions are not blocked either.

Context warnings

Now for the small new feature. In Part 1, the context compaction was triggered silently whenever the token count exceeded the threshold. So, the user had no way of seeing it coming. In the UI, we have a chat area that can show system messages cheaply, so we can do better: send a warning at 85%, 90%, and 95% of the threshold, and then run the actual compaction automatically when the limit is hit.

This goes inside `stream_to_websocket`, right at the start:

```
async def stream_to_websocket(agent, websocket):
    tokens = agent.estimate_tokens()
    for threshold_pct, label in [(85, "15%"), (90, "10%"), (95, "5%")]:
        if tokens >= CONTEXT_THRESHOLD * threshold_pct // 100 and threshold_pct
            agent._warned_thresholds.add(threshold_pct)
            await websocket.send_json({
                'type': 'system',
                'content': f"Warning: {label} context space remaining ({tokens}
            })
    if tokens > CONTEXT_THRESHOLD:
        await websocket.send_json({
            'type': 'system', 'content': f"Compacting context... ({tokens}/{CON
        await agent.compact_history()
        await websocket.send_json({'type': 'system', 'content': 'Context compac
    # ... then the rest of the streaming logic from Stage 4
```

Here is a step-by-step description:

The warning loop:

- For each threshold (85%, 90%, 95%), check if the current token count is above it.
- If yes, and if we have not warned at this threshold yet, send a system message and remember that we did.
- The `_warned_thresholds` set on the agent prevents the same warning from being sent on every turn. Without this, the user would see "Warning: 15% remaining" repeated on every message once they were close to the limit.

The auto-compaction:

- If the token count is above the threshold, send a system message, then run the compaction (asynchronously, so the event loop stays free), then send another system message when it is done.
- The compaction clears `_warned_thresholds`, so the next time the user gets close to the limit they will see the warnings again.

So, with this in place, the user sees what is going on with the context window without having to click `/context` themselves. And if the limit is hit, the UI tells them, runs the compaction in the background, and continues. So, it is much less mysterious than the silent CLI version.

One small bonus: the same auto-compaction logic also fires from the `/context` button. So, clicking the button when you are over the threshold both shows you the token count and triggers a compaction. This way, the button is a single-shortcut for "give me a status check and clean up if needed".

Running the final version

So, to run the complete agent:

```
mkdir my-session && cd my-session  
chat-ui-stage7
```

Then open `http://127.0.0.1:8000/` in your browser. The `my-session` folder is where the `history/` and `skills/` folders will be created. So, if you want to share skills across runs, drop the `.md` files into `my-session/skills/` before starting.

Screenshot of the browser window running stage 7, showing available commands and compaction in action

A few things to note: In the CLI, the slash commands also included `/history-list`, `/history-load`, `/loop`, and `/stop-loop`. The first two are obsolete now, since the sessions sidebar replaces them. The loop commands are a bit more involved (the loop needs to push messages back to the browser asynchronously), so I have left them out of this article. If you want to add them, the trick is to have the background loop send to the WebSocket using `asyncio.run_coroutine_threadsafe`. So, you may want to check that out as well.

Putting it all together

Putting all of this together, we end up with one `main.py` and one `index.html` per stage, organized as a small Python package with `pyproject.toml`. The seven `chat-ui-stageN` console scripts give you a quick way to jump between stages without remembering `uvicorn` flags.

What we have:

- A two-pane chat UI in the browser, with no build step.
- A WebSocket connection that streams thinking, content, tool calls, and tool results in real time.
- Multiple concurrent sessions, each with its own message history.
- A skills picker in the sidebar that loads personas instantly.
- Slash commands as buttons, for direct control over the session.
- Context warnings at 85%, 90%, and 95%, with automatic compaction when the limit is hit. The compaction runs in a thread, so the UI stays responsive.

The total size of the final stage is around 280 lines of Python and 150 lines of HTML+CSS+JS. So, you can read the whole thing in one sitting, which is the point.

There is also still plenty of room to improve and extend this. For example: improved session management, unloading of skills, rendering markdown in

the assistant bubbles (a few lines with [marked.js](#)), exposing the agent over a public URL using something like [Cloudflare Tunnel](#), web search with SearXNG, or adding voice input via the browser's Speech Recognition API.

The [OpenClaw](#) guide that inspired this whole series moves on to event-driven architectures and multi-agent routing in its Phase 2. So, if you want to keep going, that is a good place to look next.

For me, the value of building this from scratch is the same as it was in Part 1: I now know exactly what each piece does, and the abstractions in heavier frameworks (FastAPI starter templates, Open WebUI, LobeChat, and so on) make a lot more sense once you have built the thin version yourself.

References

- The full code: [jffjensen/building-a-local-LLM-agent-with-WebUI](#)
- [FastAPI WebSockets documentation](#)
- [Ollama Python library](#)
- [Build Your Own OpenClaw](#)
- [Part 1 of this series](#)

This story is published on [Generative AI](#). Connect with us on [LinkedIn](#) and

follow [Zeniteq](#) to stay in the loop with the latest AI stories.

Subscribe to our [newsletter](#) and [YouTube](#) channel to stay updated with the latest news and updates on generative AI. Let's shape the future of AI together!

[Python](#)[Ollama](#)[Fastapi](#)[LLM](#)[Skills](#)

Published in Generative AI

[Following](#)

87K followers · Last published 6 hours ago

Stay updated with the latest news, research, and developments in the world of generative AI. We cover everything from AI model updates, comprehensive tutorials, and real-world applications to the broader impact of AI on society. Work with us: jimclydegm@gmail.com



Written by Jes Fink-Jensen

[Follow](#)

724 followers · 579 following

Not your average geek...

